

Kruskal Algorithm Time Complexity:

Time Complexity: $O(E \log E)$ or $O(E \log V)$. Sorting of edges takes $O(E \log E)$ time. After sorting, we iterate through all edges and apply find-union algorithm. The find and union operations can take at most $O(\log V)$ time. So overall complexity is $O(E \log E + E \log V)$ time. The value of E can be at most $O(V^2)$, so $O(\log V)$ are $O(\log E)$ same. Therefore, overall time complexity is $O(E \log E)$ or $O(E \log V)$

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  create a single list of the edges in  $G.E$ 
5  sort the list of edges into monotonically increasing order by weight  $w$ 
6  for each edge  $(u, v)$  taken from the sorted list in order
7      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
8           $A = A \cup \{(u, v)\}$ 
9          UNION( $u, v$ )
10 return  $A$ 
```

Prim's Algorithm Time Complexity:

The running time of Prim's algorithm depends on the specific implementation of the min-priority queue Q . You can implement Q with a binary min-heap, including a way to map between vertices and their corresponding heap elements. The BUILD-MIN-HEAP procedure can perform lines 5-7 in $O(V)$ time. In fact, there is no need to call BUILD-MIN-HEAP. You can just put the key of r at the root of the min-heap, and because all other keys are infinity, they can go anywhere else in the min-heap. The body of the while loop executes $|V|$ times, and since each EXTRACT-MIN operation takes $O(\lg V)$ time, the total time for all calls to EXTRACT-MIN is $O(V \lg V)$. The for loop in lines 10-14 executes $O(E)$ times altogether, since the sum of the lengths of all adjacency lists is $2|E|$. Within the for loop, the test for membership in Q in line 11 can take constant time if you keep a bit for each vertex that indicates whether it belongs to Q and update the bit when the vertex is removed from Q . Each call to DECREASE-KEY in line 14 takes $O(\lg V)$ time. Thus, the total time for Prim's algorithm is $O(V \lg V + E \lg(V/)) = O(E \lg V)$, which is asymptotically the same as for our implementation of Kruskal's algorithm.

MST-PRIM(G, w, r)

```
1  for each vertex  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = \emptyset$ 
6  for each vertex  $u \in G.V$ 
7      INSERT( $Q, u$ )
8  while  $Q \neq \emptyset$ 
9       $u = \text{EXTRACT-MIN}(Q)$            // add  $u$  to the tree
10     for each vertex  $v$  in  $G.Adj[u]$  // update keys of  $u$ 's non-tree neighbors
11         if  $v \in Q$  and  $w(u, v) < v.key$ 
12              $v.\pi = u$ 
13              $v.key = w(u, v)$ 
14             DECREASE-KEY( $Q, v, w(u, v)$ )
```